

1. What is Data Modeling?

Data Modeling is the process of **organizing and structuring data so that it can be efficiently stored, understood, and used for analytics and reporting.**

In Data Engineering, we commonly design analytical models that make it easier to answer business questions.

For example, an e-commerce company may want to answer:

- What is the total revenue?
- Which products generate the most sales?
- Which customers generate the most revenue?
- What are monthly sales trends?
- Which cities generate the most orders?

Instead of repeatedly querying complex operational tables, we organize the data into an analytics-friendly model.

2. Operational Data vs Analytical Data

An e-commerce application may have operational tables such as:

customers
orders
order_items
products
payments

These tables are primarily designed to run the application efficiently.

For analytics, we may restructure this information into:

FACT_SALES
DIM_CUSTOMER
DIM_PRODUCT
DIM_DATE

This makes reporting and analytical queries simpler.

3. Grain

Before designing a fact table, one of the most important decisions is:

What does one row represent?

This is called the **Grain** of the fact table.

Example

Suppose one order contains:

Order 1001

Laptop

Mouse

Keyboard

If our grain is:

One row = One product in one order

then Order 1001 will generate three rows in the fact table.

Why is Grain Important?

The grain determines:

- What the fact table represents
- What measures can be stored
- How aggregations will behave
- Which dimensions can be connected

Always define the grain before designing the fact table.

4. Fact Table

A Fact Table stores **business events and measurable values**.

For our e-commerce example:

FACT_SALES

order_id
customer_key
product_key
date_key
quantity
unit_price
discount
sales_amount

Measures

Measures are numeric values that can generally be aggregated.

Examples:

quantity
sales_amount
discount
profit

For example:

SUM(sales_amount)

can calculate total revenue.

Simple Rule

Fact = What happened and how much?

5. Dimension Table

Dimension tables contain **descriptive information about the business event**.

Customer Dimension

DIM_CUSTOMER

customer_key
customer_id
customer_name

city
customer_segment

Product Dimension

DIM_PRODUCT

product_key
product_id
product_name
category
brand

Date Dimension

DIM_DATE

date_key
full_date
month
quarter
year

Simple Rule

Think of dimensions as answering questions such as:

Who? → Customer

What? → Product

When? → Date

Where? → Location

6. Fact vs Dimension

Fact	Dimension
Stores business events	Stores descriptive information
Contains measures	Contains attributes

Usually grows quickly Usually smaller than fact tables

Example: sales amount Example: product category

Example: quantity Example: customer city

Example:

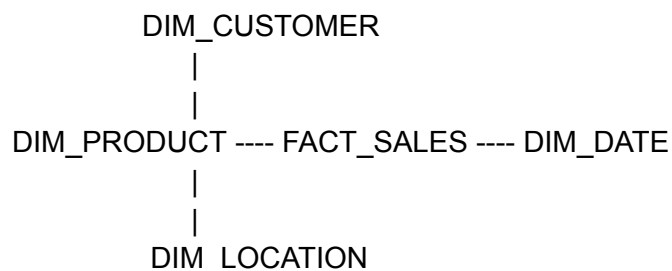
Quantity → Fact
Sales Amount → Fact

Customer Name → Dimension
Customer City → Dimension
Product Brand → Dimension

7. Star Schema

A Star Schema has a central fact table surrounded by dimension tables.

Example:



The Fact table contains the business event.

The Dimensions provide additional context.

Why Use a Star Schema?

- Simple to understand
 - Easy for analytical queries
 - Fewer joins compared with highly normalized models
 - BI-friendly structure
-

8. Surrogate Key

Suppose the source system contains:

customer_id = 101

This is a business or natural key.

Inside the warehouse, we may generate:

customer_key = 5001

customer_key is a **Surrogate Key**.

It is generated and controlled by the analytical system rather than coming directly from the source.

Example

customer_key	customer_id	customer_name
5001	101	John
5002	102	Emma

The Fact table can then reference:

customer_key

instead of depending only on the source ID.

Why Use Surrogate Keys?

They help when:

- Source IDs change
- Multiple source systems are integrated
- Historical records need to be maintained
- SCD Type 2 is implemented

9. SCD — Slowly Changing Dimensions

Dimension attributes can change over time.

For example:

Customer: John

Old City: Bengaluru

New City: Delhi

The question is:

Should we keep the old value or only the latest value?

Slowly Changing Dimensions help us handle this requirement.

10. SCD Type 1

SCD Type 1 **overwrites the existing value.**

Before:

101 | John | Bengaluru

After:

101 | John | Delhi

The old value is lost.

Use Type 1 When

Historical information is not required.

Examples:

- Correcting spelling mistakes
- Correcting invalid data
- Updating attributes where history is irrelevant

Remember

SCD Type 1 = Overwrite

11. SCD Type 2

SCD Type 2 **preserves historical information**.

Instead of updating the existing row, we expire it and create another row.

Example:

customer_key	customer_id	city	start_date	end_date	current
5001	101	Bengaluru	2025-01-01	2026-05-31	N
5002	101	Delhi	2026-06-01	NULL	Y

Now we know both:

Previous City → Bengaluru

Current City → Delhi

Common SCD Type 2 Columns

surrogate_key
business_key
effective_start_date
effective_end_date
is_current

Remember

SCD Type 1 → Overwrite history

SCD Type 2 → Preserve history

12. Practical E-Commerce Model

Suppose the business wants to analyze sales.

Grain

One row per product per order

Fact Table

FACT_SALES

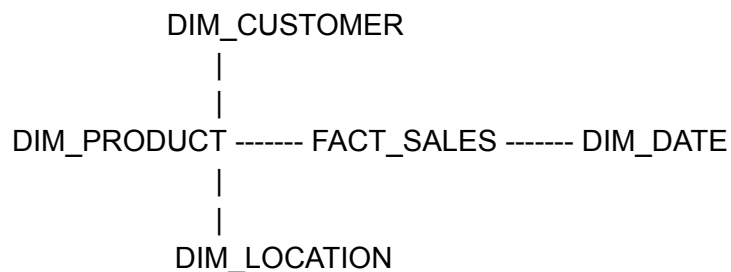
order_id

customer_key
product_key
date_key
quantity
unit_price
discount
sales_amount

Dimensions

DIM_CUSTOMER
DIM_PRODUCT
DIM_DATE
DIM_LOCATION

Model



Using this model, the business can calculate:

- Revenue by customer
- Revenue by product
- Revenue by category
- Revenue by city
- Revenue by month
- Quantity sold

13. Practical Interview Questions

Q1. Where should **quantity be stored?**

Fact Table, because quantity is a measurable value associated with the transaction.

Q2. Where should **customer_city be stored?**

Customer Dimension, because city describes the customer.

Q3. Where should `product_category` be stored?

Product Dimension, because category describes the product.

Q4. Customer changes city and business wants historical reporting. What should you use?

SCD Type 2.

Q5. Business only wants the latest customer information. What can you use?

SCD Type 1.

Q6. What should you decide before creating a fact table?

The Grain.

Q7. Why use a surrogate key?

To create a warehouse-controlled identifier that supports source-system integration and historical tracking.

14. Quick Revision

Remember this sequence while designing an analytical model:

Understand Business Requirement



Define the Grain



Identify Facts / Measures



Identify Dimensions



Define Keys



Decide Historical Requirements



Build the Model

Core Concepts

Grain

→ What does one fact-table row represent?

Fact

→ Business event + measurable values

Dimension

→ Descriptive information

Star Schema

→ Fact in center + dimensions around it

Surrogate Key

→ Warehouse-generated key

SCD Type 1

→ Overwrite

SCD Type 2

→ Preserve history

These are the fundamental Data Modeling concepts a Data Engineer should understand before moving into more advanced Data Warehousing topics.

Data with Anurag — DataX Bootcamp

For a structured Data Engineering learning path covering **SQL, Python, Data Modeling, PySpark, Big Data, Databricks, Cloud, hands-on projects, migrations, and interview preparation**:

DataX Bootcamp – Zero to Job Ready AI-Enabled Data Engineer in 90 Days

<https://login.datawithanurag.com/services/dataxbootcamp>

Bootcamp Support / Queries

<https://wa.me/918957284012>

Data with Anurag

<https://datawithanurag.com/>